Math for Data Science - Lab 5

Course: Math for Data Science

This lab puts the integral to work. We meet **continuous random variables**, where the integral stops being a technique and becomes the definition: we start with the expectation as an integral against the density, then look at the PDF and the CDF themselves, and then approximate integrals by **Monte Carlo** when the algebra defeats us. We finish with the **normal** distribution.

Exercises 1-4 are the core of the lab. Exercise 5 - the exponential and memorylessness - and exercise 6 - the integral as a limit of rectangles - are *[optional]*, and can be skipped if you are short of time.

Every question is marked with what kind of answer it wants:

- **[Code]** - the code is written for you except for one or two lines, marked `# here your code`. The comment just above the gap names the function to use. Fill it in, run the cell, and play with it. None of it asks you to write a program from scratch.
- **[Pen & paper]** - a calculation, derivation or explanation. Write it in the cell marked *here your solution* (or on paper).
- **[Code + Pen & paper]** - fill in the code, then explain what you see in the text cell below.

Setup

Run the cell below once to import everything we need.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import integrate, stats
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: Expectation of a Continuous Random Variable

A continuous random variable is described by a **probability density function** (PDF) f_X instead of a PMF. Probabilities are areas under it, $P(a < X < b) = \int_a^b f_X(x)\,dx$, so it is non-negative and integrates to 1. (Exercise 2 looks at the PDF more closely.)

The definition of expectation carries over from the discrete case with the sum replaced by an integral and the PMF by the PDF:

$$E(X) = \int_{-\infty}^{\infty} x f_X(x)\,dx, \qquad E(g(X)) = \int_{-\infty}^{\infty} g(x) f_X(x)\,dx$$

and the variance is still $\mathrm{Var}(X) = E(X^2) - (E(X))^2$.

1. [Pen & paper] Let $X \sim \mathrm{Unif}(a, b)$, whose density is $f(x) = \frac{1}{b-a}$ on $[a, b]$. Derive $E(X)$ by integration. Does the answer look right?

here your solution

2. [Pen & paper] Still with $X \sim \mathrm{Unif}(a, b)$, derive $\mathrm{Var}(X)$.

> **Hint:** compute $E(X^2)$ by integration first, then use $\mathrm{Var}(X) = E(X^2) - (E(X))^2$. The factorization $b^3 - a^3 = (b-a)(a^2 + ab + b^2)$ helps.

here your solution

3. [Pen & paper] Now rescale the variable: let $l = b - a$ be the length of the interval and define

$$Y = \frac{X - a}{l}$$

Find the distribution of Y by a change of variables.

> **Hint:** write X in terms of Y. For an increasing transformation the densities are related by $f_Y(y) = f_X(x)\,\frac{dx}{dy}$. Don't forget to work out which values Y can take.

here your solution

4. [Pen & paper] Calculate the mean and the variance of the standard uniform Y directly from its density. Then use $X = a + lY$ to get back the results of questions 1 and 2.

here your solution

5. [Pen & paper] Now let $X \sim \text{Expo}(\lambda)$, with density $f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$. Derive $E(X)$.

Hint: integrate by parts, $\int u\,dv = uv - \int v\,du$, with $u = x$ and $dv = \lambda e^{-\lambda x} dx$.

here your solution

6. [Pen & paper] *[optional]* Still with $X \sim \text{Expo}(\lambda)$, derive $\text{Var}(X)$.

Hint: compute $E(X^2)$ by parts again, now with $u = x^2$ and $dv = \lambda e^{-\lambda x} dx$. The integral that is left over is a multiple of the one you already solved in question 5.

here your solution

7. [Pen & paper] *[optional]* Normalize the exponential, as you did with the uniform in question 3: let $Y = \lambda X$. Find the distribution of Y by a change of variables, and use it to get back the mean and the variance of X.

Hint: write $X = Y/\lambda$ and use $f_Y(y) = f_X(x)\,\frac{dx}{dy}$ again. The mean and variance of Y then follow from questions 5 and 6 with $\lambda = 1$.

here your solution

8. [Code] Check the expectations from questions 1 and 5 numerically: integrate $xf(x)$ with `quad`, and compare against a large simulation. Use $\text{Unif}(2, 8)$ and $\text{Expo}(\lambda = 0.5)$.

Hint: `integrate.quad(f, a, b)` computes $\int_a^b f(x)\,dx$ numerically - it is the equivalent of R's `integrate`. It returns a pair, the estimate and an error bound, so write `value, _ = integrate.quad(...)` (the `_` throws the error bound away). The functions $xf(x)$ are written for you as `x_times_pdf_unif` and `x_times_pdf_exp`; use `np.inf` for ∞. For the simulation, `rng.uniform(low, high, size=...)` and `rng.exponential(scale=..., size=...)` draw the values and `.mean()` averages them. Careful: NumPy parameterizes the exponential by the **scale** $1/\lambda$, not the rate λ - `rng.exponential(scale=2.0)` gives you $\text{Expo}(0.5)$. R's `rexp` uses the rate, so this is a classic place to slip.

```
rng = np.random.default_rng(314)
n_sim = 1_000_000

# uniform on [2, 8]
a_u, b_u = 2, 8
```

```python
def x_times_pdf_unif(x):
    return x * 1 / (b_u - a_u)


# Two lines: `num_unif` by integrating `x_times_pdf_unif` from a_u to b_u, and
# `sim_unif`, the mean of n_sim uniform draws on [a_u, b_u].
# here your code

print("Unif(2, 8)")
print(f"  formula (a+b)/2 = {(a_u + b_u) / 2:.5f}")
print(f"  quad            = {num_unif:.5f}")
print(f"  simulation      = {sim_unif:.5f}")

# exponential with rate 0.5
lam = 0.5


def x_times_pdf_exp(x):
    return x * lam * np.exp(-lam * x)


# Two lines: `num_exp` by integrating `x_times_pdf_exp` from 0 to np.inf, and
# `sim_exp`, the mean of n_sim exponential draws (mind the scale!).
# here your code

print("\nExpo(0.5)")
print(f"  formula 1/lambda = {1 / lam:.5f}")
print(f"  quad             = {num_exp:.5f}")
print(f"  simulation       = {sim_exp:.5f}")
```

Exercise 2: Continuous Random Variables - PDF and CDF

A continuous random variable has no PMF: $P(X = x) = 0$ for every x. What it has instead is a **probability density function** f_X, which is not a probability but a density - probability per unit of x. You get probabilities out of it by integrating:

$$P(a < X < b) = \int_a^b f_X(x)\,dx, \qquad F_X(x) = P(X \le x) = \int_{-\infty}^{x} f_X(t)\,dt$$

So the CDF is the integral of the PDF, and - by the fundamental theorem of calculus - the PDF is the derivative of the CDF. To be a valid PDF, f_X must be non-negative everywhere and integrate to 1 over the whole line.

1. [Pen & paper] Consider the candidate density

$$f(x) = \begin{cases} cx^2 0 \le x \le 3 \\ 0 \quad \text{otherwise} \end{cases}$$

Find the value of c that makes this a valid PDF. Then find F_X and use it to compute $P(1 < X < 2)$.

here your solution

2. [Code] Verify all of this in code: check that the density integrates to 1, plot the PDF and the CDF side by side, and confirm $P(1 < X < 2)$ three ways - from your F_X formula, by numerical integration of the PDF, and by simulation.

> **Hint:** `integrate.quad(pdf, a, b)` integrates the PDF from `a` to `b`. To simulate from this distribution, use **inverse transform sampling**: if $U \sim \text{Unif}(0, 1)$ then $F_X^{-1}(U)$ has CDF F_X. Here $F_X(x) = x^3/27$, so $F_X^{-1}(u) = 3u^{1/3}$ - in Python, `u ** (1 / 3)`. The share of draws between 1 and 2 is `np.mean((draws > 1) & (draws < 2))`.

```
def pdf(x):
    return np.where((x >= 0) & (x <= 3), x ** 2 / 9, 0.0)


def cdf(x):
    x = np.clip(x, 0, 3)          # outside [0, 3] the CDF is flat at 0 or 1
    # Return your formula for F_X(x).
    # here your code


# Integrate `pdf` over its support [0, 3]; the result should be 1.
# here your code
print(f"integral of the density over its support = {total:.6f}")

# --- plotting, written for you ---
grid = np.linspace(-0.5, 3.5, 400)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(grid, pdf(grid), color="blue")
axes[0].fill_between(grid, 0, pdf(grid), where=(grid >= 1) & (grid <= 2),
                     color="blue", alpha=0.3)
axes[0].set_title("PDF, shaded on $1 < x < 2$")
axes[1].plot(grid, cdf(grid), color="red")
axes[1].set_title("CDF $F_X(x) = x^3/27$")
for ax in axes:
    ax.set_xlabel("$x$")
fig.tight_layout()
plt.show()
```

```
rng = np.random.default_rng(5)
u = rng.uniform(0, 1, size=500_000)

# Three ways to get P(1 < X < 2), one line each:
#   `formula`   - F(2) - F(1), using `cdf`
#   `numeric`   - integrate `pdf` from 1 to 2 with `integrate.quad`
#   `draws`     - turn the uniforms `u` into draws of X with the inverse CDF 3
* u^(1/3)
#   `simulated` - the proportion of `draws` between 1 and 2
# here your code

print(f"\nfrom F(2) - F(1)    = {formula:.5f}")
print(f"numerical integral  = {numeric:.5f}")
print(f"simulation          = {simulated:.5f}")
print(f"exact 7/27          = {7 / 27:.5f}")
```

3. [Pen & paper] A PDF is *not* a probability, and one consequence catches everyone out the first time: a density may be **greater than 1**. Give an example, and explain why that is not a contradiction.

here your solution

Exercise 3: Monte Carlo Integration - Throwing Darts

Although we now understand integration conceptually, we have not done much of it yet (outside of nice polynomial functions) because computing integrals by hand gets complicated quickly. Here is a computational method for approximating integrals that uses probability: **Monte Carlo integration**.

Let f be a complicated function whose integral $\int_a^b f(x)\,dx$ we want to approximate. Assume that $0 \leq f(x) \leq c$, so that we know the integral is finite.

On the surface this problem doesn't involve probability, as $\int_a^b f(x)\,dx$ is just a number. But if we cannot find that number using exact integration methods, we will use random samples to obtain an approximation.

Let A be the rectangle given by $a \leq x \leq b$ and $0 \leq y \leq c$. Let B be the region under the curve $y = f(x)$ (and above the x-axis) for $a \leq x \leq b$. To find the area of the region B, we take random samples from A and compute the proportion of those samples that also fall into the region B.

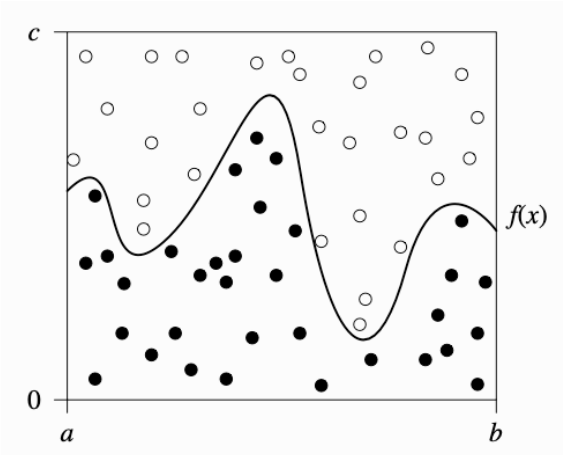


Figure 1: Monte Carlo Integration

To see why this works, suppose we pick i.i.d. points $(X_1, Y_1), ..., (X_n, Y_n)$ uniformly on the rectangle A. Define indicator random variables $I_1, ..., I_n$ by letting $I_j = 1$ if (X_j, Y_j) is in B and $I_j = 0$ otherwise. Then the I_j are Bernoulli random variables whose success probability is the ratio of the area of B to the area of A:

$$p = P(I_j = 1) = E(I_j) = \frac{\int_a^b f(x)\,dx}{c(b-a)}$$

We can estimate p using $\frac{1}{n}\sum_{j=1}^{n} I_j$, and then estimate the desired integral by

$$\int_a^b f(x)\,dx \approx c(b-a)\,\frac{1}{n}\sum_{j=1}^{n} I_j$$

Let's try this out in code for the function

$$f(x) = \left(\frac{1}{\sqrt{x+2}}\right)^{xe+x^3} + \frac{x^3}{5}$$

1a. [Code] First, put this function into Python as `monte_fun`.

> **Hint:** `np.sqrt(...)` is the square root, `np.e` is the number e, and `**` is "to the power of" (so x^3 is `x ** 3`).

```
def monte_fun(x):
    """The function f(x) we want to integrate."""
    # Return f(x), translated from the formula above.
    # here your code
```

1b. [Code] Quickly plot this function over the interval $x \in [0, 5]$.

Hint: `ax.plot(x_values, y_values)` draws a line. The x-values are already in `grid`.

```
grid = np.linspace(0, 5, 400)

fig, ax = plt.subplots(figsize=(7, 4.5))
# Plot `monte_fun` against `grid` with `ax.plot`.
# here your code
ax.set_xlabel("x")
ax.set_ylabel("y")
fig.tight_layout()
plt.show()
```

1c. [Code] Create a data frame containing a uniform random sample of 1000 points inside the rectangle $x \in [0, 5], y \in [0, 25]$.

Hint: `rng.uniform(low, high, size=1000)` draws 1000 uniform numbers between `low` and `high`.

```
rng = np.random.default_rng(123)

# Two lines: 1000 uniform x-coordinates in [0, 5] and 1000 uniform y-coordinates
in [0, 25].
# here your code

points = pd.DataFrame({"x": x_points, "y": y_points})
points.head()
```

1d. [Code] Plot these points on a graph together with our function.

Hint: `ax.scatter(x, y, s=8, color="black")` draws points; `ax.plot(x, y, color="red")` draws the curve. The columns of the data frame are `points["x"]` and `points["y"]`.

```
fig, ax = plt.subplots(figsize=(7, 4.5))
# Two lines: a scatter of the points, and a line plot of `monte_fun` over `grid`.
# here your code

# --- plotting, written for you ---
ax.set_xlim(0, 5)
ax.set_ylim(0, 25)
ax.set_xlabel("x")
```

```
ax.set_ylabel("y")
fig.tight_layout()
plt.show()
```

1e. [Code] Now create a column in your data frame that indicates whether the point is below the curve (1 for below, 0 for above).

> **Hint:** a point is below the curve when its y is smaller than `monte_fun` of its x. The comparison `a < b` gives `True`/`False`; add `.astype(int)` to turn that into `1`/`0`.

```
# Create the column `points["i"]`: 1 if the point is below the curve, 0 otherwise.
# here your code
points.head()
```

1f. [Code] Create a new graph like the one in part (d), but color the points below the curve in red.

> **Hint:** `np.where(condition, "red", "black")` picks `"red"` wherever the condition is true and `"black"` elsewhere. The condition is `points["i"] == 1`.

```
# One color per point: red if it is below the curve, black otherwise.
# here your code

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.scatter(points["x"], points["y"], s=8, c=colors)
ax.plot(grid, monte_fun(grid), color="red")
ax.set_xlim(0, 5)
ax.set_ylim(0, 25)
ax.set_xlabel("x")
ax.set_ylabel("y")
fig.tight_layout()
plt.show()
```

1g-h. [Code] Find p, or $E(I_j)$. Then find the approximation to the integral of our function over $x \in [0, 5]$.

> **Hint:** the average of a column is `points["i"].mean()`. The integral estimate is the area of the rectangle, $c(b-a)$, times $\hat{p}$.

```
c, a, b = 25, 0, 5

# Two lines: `p_hat`, the proportion of points below the curve, and `estimate`,
# the area of the rectangle times `p_hat`.
# here your code

print(f"p_hat = {p_hat:.4f}")
print(f"Monte Carlo estimate = {estimate:.4f}")
```

1i-j. [Code] Put all of the Monte Carlo integration steps above into a function called `mcint` and run it for a few different values of n. Then compute the true answer with `scipy.integrate.quad` (or Wolfram Alpha) and compare.

> **Hint:** these are the same steps as (e) and (h) - `y < monte_fun(x)` for the indicator, `.mean()` for $\hat{p}$ - and `integrate.quad(f, a, b)` as in Exercise 1.

```
def mcint(n, seed=123):
    rng = np.random.default_rng(seed)
    x = rng.uniform(0, 5, size=n)
    y = rng.uniform(0, 25, size=n)
    # Two lines: the indicator `i` (is y below the curve?), and the return value,
    # the rectangle area 5 * 25 times the mean of `i`.
    # here your code


for n in (1_000, 10_000, 100_000, 1_000_000):
    print(f"mcint(n={n:>9,}) = {mcint(n):.4f}")

# Compare to a deterministic quadrature routine: integrate `monte_fun` from 0
to 5.
# here your code
print(f"\nscipy.integrate.quad  = {exact:.4f}  (estimated error {err:.2e})")
```

Exercise 4: The Normal Distribution

The normal density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\, e^{-(x-\mu)^2/(2\sigma^2)}$$

with μ the **location** parameter (where it is centred) and σ^2 the **scale** parameter (how spread out it is). Its most useful property is **standardization**: if $X \sim \mathcal{N}(\mu, \sigma^2)$, then

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1)$$

so every normal question can be turned into a question about the standard normal.

1. [Pen & paper] Suppose adult height is $\mathcal{N}(\mu = 170, \sigma^2 = 10^2)$ cm. Compute $P(X > 185)$ by hand, by standardizing and then looking up Φ.

here your solution

2. [Code] Confirm it in code, and check the **68-95-99.7 rule**: the proportion of the distribution falling within 1, 2 and 3 standard deviations of the mean. Verify both by the CDF and by simulation.

> **Hint:** `stats.norm.cdf(x, loc=mu, scale=sigma)` is $P(X \leq x)$. It takes the standard *deviation* σ as `scale`, not the variance. Without `loc` and `scale` it is the standard normal Φ. `rng.normal(mu, sigma, size=...)` draws normal values.

```
mu, sigma = 170, 10
rng = np.random.default_rng(1908)

# Two lines: `p_tall`, P(X > 185) from `stats.norm.cdf`, and `heights`,
# 1,000,000 simulated heights from `rng.normal`.
# here your code

print(f"P(X > 185) = {p_tall:.4f}")
print(f"simulated  = {np.mean(heights > 185):.4f}")

print("\nthe 68-95-99.7 rule:")
for k in (1, 2, 3):
    # `exact` = P(-k < Z < k) = Phi(k) - Phi(-k), using the standard normal
`stats.norm.cdf`.
    # here your code
    sim = np.mean(np.abs(heights - mu) < k * sigma)
    print(f"  within {k} sd:  exact {exact:.4f}   simulated {sim:.4f}")
```

3. [Code] *[optional]* Standardization is a claim about a *transformation* of a random variable. Check it directly: simulate $X \sim \mathcal{N}(170, 10^2)$, form $Z = (X - \mu)/\sigma$, and compare the histogram of Z against the standard normal density.

> **Hint:** `heights` from question 2 already holds the simulated X. Arithmetic on an array works element by element, so the formula can be typed as it stands.

```
# Standardize the simulated heights: Z = (X - mu) / sigma.
# here your code

# --- plotting, written for you ---
grid = np.linspace(-4, 4, 400)
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(Z, bins=100, density=True, color="steelblue", alpha=0.5, label="$(X - \
\mu)/\\sigma$")
ax.plot(grid,   stats.norm.pdf(grid),   color="red",   linewidth=2,   label="$\
\mathcal{N}(0,1)$ density")
ax.set_xlabel("$z$")
ax.set_ylabel("density")
ax.legend()
fig.tight_layout()
plt.show()

print(f"mean of Z = {Z.mean():.4f}  (target 0)")
print(f"sd of Z   = {Z.std():.4f}  (target 1)")
```

Exercise 5: The Exponential and Memorylessness

[optional]

The exponential distribution has a property no other continuous distribution on $[0, \infty)$ has: it is **memoryless**. For all $s, t > 0$,

$$P(X > s + t \mid X > s) = P(X > t)$$

In words: given that you have already waited s minutes, the distribution of your *remaining* wait is exactly what it was when you started. The process does not "age".

1. [Pen & paper] Prove it. Start from the survival function $P(X > t) = e^{-\lambda t}$, which you can get from the CDF $F(t) = 1 - e^{-\lambda t}$.

here your solution

2. [Code] Check it by simulation with $\lambda = 0.5$ (so the mean wait is 2 minutes). Draw many waiting times, keep only those exceeding $s = 3$ minutes, subtract 3, and compare the resulting distribution to a fresh Expo(0.5) sample.

> **Hint:** `rng.exponential(scale=1 / lam, size=...)` draws waiting times. `waits[waits > s]` keeps only the waits longer than `s`.

```
rng = np.random.default_rng(1888)
```

```
lam, s = 0.5, 3.0
waits = rng.exponential(scale=1 / lam, size=1_000_000)

# Two lines: `remaining`, the waits longer than s with s subtracted, and `fresh`,
# a new exponential sample of the same length (`size=len(remaining)`).
# here your code

print(f"kept {len(remaining):,} of {len(waits):,} draws")
print(f"mean remaining wait = {remaining.mean():.4f}")
print(f"mean fresh wait     = {fresh.mean():.4f}")
print(f"theoretical 1/lam   = {1 / lam:.4f}")

# --- plotting, written for you ---
grid = np.linspace(0, 15, 300)
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.hist(remaining, bins=100, density=True, range=(0, 15), color="gray",
        alpha=0.5, label=f"remaining wait, given $X > {s:.0f}$")
ax.plot(grid, lam * np.exp(-lam * grid), color="red", linewidth=2,
        label="Expo(0.5) density")
ax.set_xlabel("minutes")
ax.set_ylabel("density")
ax.legend()
fig.tight_layout()
plt.show()
```

3. [Pen & paper] Why is memorylessness often an *unrealistic* assumption? Give an example where it clearly fails.

here your solution

Exercise 6: The Definite Integral as a Limit of Rectangles

[optional]

The definite integral $\int_a^b f(x)\,dx$ is defined as the limit of a **Riemann sum**: chop $[a, b]$ into n strips of width $\Delta x = (b-a)/n$, approximate the area of each strip by a rectangle, and let $n \to \infty$. Using the left endpoint of each strip for the rectangle's height,

$$\int_a^b f(x)\,dx = \lim_{n\to\infty} \sum_{i=0}^{n-1} f(a + i\,\Delta x)\,\Delta x$$

1. [Code] Complete the function `left_riemann(f, a, b, n)` so that it returns this sum. The cell then uses it to approximate $\int_1^4 (x^2+1)\,dx$ for $n = 4, 10, 100, 10\,000$.

Hint: `x_left` already holds all n left endpoints. `f(x_left)` gives all the heights at once, and `np.sum(...)` adds up an array.

```
def left_riemann(f, a, b, n):
    dx = (b - a) / n
    x_left = a + np.arange(n) * dx
    # Return the sum of (height x width) over all the rectangles, using `np.sum`.
    # here your code


def g(x):
    return x ** 2 + 1


for n in (4, 10, 100, 10_000):
    print(f"n = {n:>6}:  left Riemann sum = {left_riemann(g, 1, 4, n):.6f}")
```

2. [Code] Draw the picture for $n = 4$ and $n = 20$: plot f and the rectangles side by side, so you can see the approximation improve.

Hint: the left endpoints are $a + i\,\Delta x$ for $i = 0, \ldots, n-1$ - exactly the line inside `left_riemann` above. The height of each rectangle is `g` evaluated at those endpoints.

```
grid = np.linspace(1, 4, 400)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, n in zip(axes, (4, 20)):
    dx = (4 - 1) / n
     # Two lines: the left endpoints `x_left` (use `np.arange(n)`), and the
rectangle
    # heights `heights` (apply `g` to `x_left`).
    # here your code

    # --- plotting, written for you ---
    ax.bar(x_left, heights, width=dx, align="edge",
           color="steelblue", alpha=0.4, edgecolor="steelblue")
    ax.plot(grid, g(grid), color="black", linewidth=2)
    ax.set_title(f"$n = {n}$,  sum $= {left_riemann(g, 1, 4, n):.3f}$")
    ax.set_xlabel("$x$")
axes[0].set_ylabel("$f(x)$")
fig.tight_layout()
plt.show()
```

3. [Pen & paper] Now compute the integral exactly, by hand, using the **fundamental theorem of calculus**: if F is any antiderivative of f, then $\int_a^b f(x)\,dx = F(b) - F(a)$.

Compare your answer to the $n = 10\,000$ Riemann sum. Are the rectangles approaching it from above or from below, and why?

here your solution

4. [Code] Check both answers against a numerical integrator.

> **Hint:** `integrate.quad(f, a, b)`, as in Exercise 1. It returns a pair: the estimate and an error bound, so write `value, err = integrate.quad(...)`.

```
# Integrate `g` from 1 to 4 with `integrate.quad`, storing the pair in `value, err`.
# here your code

print(f"scipy.integrate.quad = {value:.10f}   (error bound {err:.1e})")
print(f"exact (FTC)          = 24")
print(f"left Riemann, n=1e5  = {left_riemann(g, 1, 4, 100_000):.6f}")
```